SIIT Study Hub

Presentation Prep Script & Q&A Rehearsal

CSS326 Database Programming Laboratory — Week 7 Proposal

Touch-ing Yinyong (6722782164) Pinyapach Sinlapakantang (6722771886)
Yayika Tinner (6722781026)

Prepared for the Week 7 proposal presentation

Purpose of this document. This is not slide content. It is what each of us reads the night before (and skims five minutes before walking in) so that everyone — not just whoever designed a given piece — can explain the whole project, defend a design decision on the spot, and answer a question about a part we didn't personally build. If you remember one thing from this document, make it Section 8 (Anticipated Q&A).

Contents

1 The 60-Second Pitch **2**
1.1 Why this matters as a database project, specifically 2

2 Tech Stack Cheat Sheet **2**
2.1 If asked "why MySQL and not Postgres / MongoDB / SQLite" 3

3 Architecture Walkthrough: What Happens on a Request **3**
3.1 The "defense in depth" idea – expect this question 3

4 Database Design Deep-Dive **4**
4.1 The nine tables 4
4.2 The ISA (supertype/subtype) design – the core modeling decision 4
4.3 Relationship sets, in plain language 4
4.4 The normalization story – a genuine "we learned from a real bug" beat 5
4.5 Why `file_path` lives on `Content`, not just `Materials` 5

5 Procedures, Functions & Triggers — What and Why **5**
5.1 Triggers 5
5.2 Stored procedures & functions 5
5.3 One-line answer to "why not just do this in Express/JavaScript?" 6

6 Physical Design Notes **6**

7 User Flow Walkthrough (know this cold) **7**

8 Anticipated Q&A **7**

8.1 Tech stack & architecture . . . 7
8.2 Database design . . . 8
8.3 Procedures, triggers & transactions . . . 8
8.4 Security . . . 9
8.5 Scope & what's not done yet . . . 9
8.6 Team & process . . . 10

9 Work Distribution & Timeline **10**

10 Glossary (fast lookup if a term catches you off guard) **11**

1 The 60-Second Pitch

Say this cold, without slides, if someone asks "what are you building":

> *"Every year, SIIT students lose institutional knowledge about a course the moment the students who took it graduate or move on — what the midterm actually covers, how hard it really is, what to study. Right now that knowledge lives and dies in LINE groups, senior hand-offs, and Google Drive links that rot between cohorts. SIIT Study Hub is a SIIT-gated web archive where students submit structured exam intelligence — format, topics, difficulty, allowed aids — plus study materials, organized by course. We're not redistributing actual exam papers, which would be a copyright and academic-integrity problem; we're capturing structured descriptions of exams. It's a from-scratch MySQL and Node.js/Express redesign of an idea that already exists as a live Postgres/Supabase product for a different course, rebuilt here with a cleaner schema based on real problems we found auditing that prototype."*

1.1 Why this matters as a database project, specifically

This is a database course, so if the pitch above is all you say, you've under-answered. The follow-up sentence should always be: *"the interesting database problems are: modeling one submission that's either an exam report or a study material without duplicating logic, keeping integrity rules that MySQL's table-level `CHECK` can't express, and moving trust-sensitive operations like moderation into the database layer instead of the application."* That sentence previews Sections 4–6 below and is what actually gets graded.

2 Tech Stack Cheat Sheet

If asked "what's your stack" – answer in this order (layer, then technology, then the one-line why):

Layer	Technology	Why
Database	MySQL 8.x	Fixed by the course track ("MySQL & Node.js (RDBMS)"); run locally, dev DB at `localhost`.
Driver	`mysql2/promise`	Promise-based, supports prepared statements (parameterized queries – prevents SQL injection) and `CALL` for stored procedures.
Connection strategy	`mysql.createPool(...)`	A pool, not a single connection – reuses connections across concurrent requests instead of opening/closing one per request.

Layer	Technology	Why
Backend framework	Node.js + Express	Required by the course; thin routing layer over the driver.
Backend structure	`routes/*.route.js` + `controller/*.controller.js`	Routes just map HTTP verb + path to a controller function; controllers hold the actual query/procedure-call logic. Keeps HTTP concerns separate from data-access logic.
Frontend	Static HTML/CSS/JS via `express.static`, from `public/`	No separate frontend framework required by the course; server-rendered/static pages calling the Express API.
Transactions	`BEGIN` / `COMMIT` / `ROLLBACK` inside stored procedures	Introduced Week 10; used specifically where a submission touches two tables (Content + subtype) that must succeed or fail together.
Semi-structured data	`JSON` columns + X DevAPI (document-store module)	Week 12 addition; `format/topics` on `ExamIntel` are declared `JSON` now, but querying them via the document-store API is deliberately deferred.

2.1 If asked "why MySQL and not Postgres / MongoDB / SQLite"

Direct answer: *the track is fixed by the course* – CSS326's "Revised Track" requires MySQL & Node.js. But there is a real answer underneath the logistics one, and it's worth giving: a live Postgres/Supabase version of this same product idea already exists (see the Q&A entry "What's the difference between this and the already-existing SIIT Study Hub product?" in Section 8.2), so this project is intentionally a chance to redo the schema on a different RDBMS with different constraint capabilities (e.g. MySQL's single-table `CHECK`) and apply lessons learned, not a case where the team picked MySQL freely.

3 Architecture Walkthrough: What Happens on a Request

Walk through this mentally for *any* user action – it is the same shape every time:

1. **Browser** sends an HTTP request (e.g. `POST /content`) to Express.
2. **Route** (`routes/content.route.js`) matches the verb + path and calls a controller function – it does no logic itself.
3. **Controller** (`controller/content.controller.js`) validates input shape, then either runs a parameterized SQL query *or* calls a stored procedure via `CALL sp_...(?, ?, ...)` through the `mysql2` pool.
4. **MySQL** executes the query/procedure. If it's a procedure, MySQL-side logic (validation, transaction, trigger firing) happens *inside the database*, not in Node.
5. **Response** flows back up: MySQL → mysql2 pool → controller (shapes JSON) → route → browser.

3.1 The "defense in depth" idea – expect this question

A sharp question a grader might ask: *"you validate the SIIT email domain in the app layer – why also enforce it with a trigger? Isn't that redundant?"*

Answer: it's redundant *by design.* App-layer validation is for good UX (fast, friendly error messages). The database-layer trigger (`trg_enforce_siit_domain`) is the actual guarantee – it's what stops a bad row from ever existing, regardless of whether the write came through our Express app, a future second client, a script, or a bug that skipped the app check. The same logic applies to moderation authorization living in `sp_moderate_content` rather than only in an Express middleware: a compromised or buggy client can't forge a takedown because the check is enforced where the write actually happens.

4 Database Design Deep-Dive

4.1 The nine tables

Table	What it holds
`User`	Account: SIIT email (domain-verified), handle, salted+hashed password, `is_admin` moderator flag.
`Course`	A course catalog entry: code, title, program, credits.
`Instructor`	Just a name/id – who can teach a `CourseOffering`.
`CourseOffering`	*(new table)* A specific (course, instructor, semester) combination.
`Content`	*Supertype.* Shared fields for any submission: author, course offering, optional file attachment, moderation fields, timestamps.
`ExamIntel`	*Subtype of Content.* Exam type, format (JSON), topics (JSON), allowed aids, duration, time pressure, difficulty, advice.
`Materials`	*Subtype of Content.* Title, type, markdown body.
`Vote`	One row per (user, content) up/down vote.
`Report`	One row per (user, content) report, with resolution state.

4.2 The ISA (supertype/subtype) design – the core modeling decision

`Content ISA {ExamIntel, Materials}`, **disjoint and total**: every `Content` row is *exactly one* of the two subtypes, never both, never neither. In plain English: one drop box (`Content`) with two flavors (`ExamIntel` for "what was on the test", `Materials` for "what helps you study for it"). `content_type` is the discriminator column (`ENUM('exam_intel','materials')`), and each subtype table shares its primary key with `Content` (`content_id` is both PK and FK in `ExamIntel`/`Materials`).

Why not two completely separate tables with no shared supertype? Because `Vote` and `Report` need to point at "a submission" regardless of which kind it is. Without a shared supertype, a vote or report would need two nullable foreign keys (one to `ExamIntel`, one to `Materials`) with no real constraint stopping both or neither from being set – exactly the "no real foreign key" bug found auditing the old prototype. With `Content` as the single target, `Vote.content_id` and `Report.content_id` are simple, honest foreign keys.

4.3 Relationship sets, in plain language

- A `User` **authors** many `Content` rows (1:M).
- `User`–`Vote`–`Content` is **M:N** (a user can vote on many submissions; a submission gets votes from many users), with `value` (+1/-1) as the relationship's own attribute.

- `User`–`Report`–`Content` is **M:N** the same way, with `reason` and `resolved` as attributes.
- A `Course` **has** many `CourseOfferings` (1:M); an `Instructor` **teaches** many `CourseOfferings` (1:M); a `CourseOffering` **has** many `Content` rows (1:M).

4.4 The normalization story – a genuine "we learned from a real bug" beat

The old working prototype stored instructor name and semester as **duplicated free text on every single submission**. That's a classic normalization smell: the same fact ("Dr. A taught CSS326 in 2026/1") repeated on hundreds of rows, capable of drifting out of sync (typo in one row, correct in another). `CourseOffering` fixes this by pulling (course, instructor, semester) into its own table with a uniqueness constraint on `(course_id, semester, instructor_id)`, and every `Content` row references *one* `CourseOffering` instead of repeating the text. This is worth telling as a story if asked "what would you do differently from the prototype" – it's a concrete, defensible answer instead of a vague "we'd make it better."

4.5 Why `file_path` lives on `Content`, not just `Materials`

First instinct was to put the optional file attachment only on `Materials` (notes/cheat sheets are files, right?). But a student attaching a scanned cheat sheet *to an exam-intel report* is just as real a case. So `file_path` moved up to the shared supertype, `Content`. **The tradeoff, and the actual interesting database point:** MySQL's `CHECK` constraint can only reference columns within a single table. The rule "a Materials row needs either body text or an attached file" now spans two tables (`Content.file_path` and `Materials.body_md`), so it *cannot* be a table-level `CHECK` anymore. That validation moved into `sp_submit_content` instead. **This is a genuinely good answer if asked "give an example of a constraint that has to live in application/procedure logic instead of the schema."**

5 Procedures, Functions & Triggers — What and Why

5.1 Triggers

Name	Fires on	What it does, and why
`trg_vote_score`	AFTER INSERT/DELETE on `Vote`	Updates a denormalized helpfulness score on the parent `Content` row. *Why:* so the course page can sort/filter by score without re-aggregating all votes on every read.
`trg_enforce_siit_domain`	BEFORE INSERT/UPDATE on `User`	Rejects any row whose email isn't on the SIIT domain. *Why:* a second, unbypassable copy of the app's own rule – a bug or a second write path can't sneak around it.
`trg_stamp_resolution`	BEFORE UPDATE on `Report`	When `resolved` flips true, stamps `resolved_at`/`resolved_by` itself. *Why:* the app never sets those columns directly, so a moderation record can't be forged by a compromised client.

5.2 Stored procedures & functions

Name	What it does, and why
`sp_register_user(email, handle, password)`	Validates SIIT domain + password policy, generates a random salt, hashes the password, inserts `User`. *Why:* keeps the hashing/salting logic (a security-critical operation) in one place, callable the same way from any client.
`sp_submit_content(...)`	Inserts `Content` and its subtype row (`ExamIntel` or `Materials`) inside one `BEGIN/COMMIT/ROLLBACK` transaction; for `materials`, also checks that `body_md` or `file_path` is present. *Why a transaction:* without it, a crash between the two inserts leaves an orphaned `Content` row with no subtype. *Why the check lives here:* see Section 4.5 – it can't be a table `CHECK` once the two columns are on different tables.
`sp_moderate_content(content_id, admin_id, remove, note)`	Checks the caller's `is_admin` flag, then soft-deletes (or restores) the `Content` row and auto-resolves open reports against it. *Why:* centralizes moderation authorization in the database instead of trusting the application layer alone – see the "defense in depth" box in Section 3.1.
`fn_course_summary(course_id)`	Returns aggregate stats for a course (avg. difficulty, avg. time pressure, most-cited topics), computed in SQL. *Why a function, and why in SQL:* every screen that needs these stats gets a single consistent computation instead of each caller re-implementing the aggregation client-side.

5.3 One-line answer to "why not just do this in Express/JavaScript?"

Because the three things procedures/triggers buy you – **atomicity** (all-or-nothing across two tables), **integrity** (a rule that's enforced no matter which client writes), and **automation** (a side effect like a score update or a timestamp that should never depend on the caller remembering to do it) – are exactly the guarantees that live at the database layer, closer to the data than any one application client.

6 Physical Design Notes

- **ENUM** is used for closed, small vocabularies known in advance: `content_type` (`exam_intel`/`materials`), `exam_type` (`midterm`/`final`/`quiz`), `allowed_aids`, `Materials.type`. Cheap to store, self-documenting, and the database rejects an invalid value outright.
- **JSON** is used for `ExamIntel.format` and `ExamIntel.topics` – both are variable-length lists (an exam can be MCQ *and* written; it can cover an open-ended set of topics) that don't fit a fixed ENUM or a separate join table cleanly at this stage. The column type is declared now; querying it through MySQL's document-store (X DevAPI) module is a **Week 12** deliverable, not done yet – if asked "is the JSON/NoSQL part done," the honest answer is *"the column exists, the document-store querying work is scheduled for Week 12 per the syllabus."*
- **Soft delete**, not hard delete, for moderation: `Content` has `removed_at`/`removed_by`/`removed_note` instead of a `DELETE` statement. This preserves the audit trail (who removed what, when, why) and lets `sp_moderate_content` *restore* content – impossible if the row were actually deleted.

- **Unique constraints** worth knowing cold: `CourseOffering(course_id, semester, instructor_id)`, `Vote(content_id, user_id)` (one vote per user per submission), `Report(content_id, reporter_id)` (one open report per user per submission).
- **Password storage**: `password_hash` (VARCHAR) + `salt` (VARBINARY) as separate columns, set only by `sp_register_user` – never plaintext, never touched directly by the app layer.

7 User Flow Walkthrough (know this cold)

Each step names the concrete DB operation – this is the single best thing to have memorized, because it's the fastest way to answer "walk me through what happens when a user does X."

1. **Sign up.** Email + handle + password → app checks domain/password policy → `CALL sp_register_user(...)` salts, hashes, inserts `User`.
2. **Browse courses.** Search/filter by code or title → indexed `SELECT` on `Course`.
3. **View a course page.** `SELECT` across `CourseOffering` and `Content` (joined to its subtype); aggregate stats from `fn_course_summary(course_id)`.
4. **Submit exam intelligence.** Form (format, topics, difficulty, advice) → `CALL sp_submit_content(...)` inserts `Content` + `ExamIntel` in one transaction.
5. **Submit study material.** Notes and/or file → same procedure inserts `Content` + `Materials`, checking body text or file is present.
6. **Edit/delete own submission.** `UPDATE`/`DELETE` on `Content` + subtype, scoped to `author_id`, inside a transaction.
7. **Vote.** `INSERT`/`UPDATE` on `Vote` → `trg_vote_score` updates the denormalized score.
8. **Report.** `INSERT` into `Report`.
9. **Moderate.** `CALL sp_moderate_content(...)` soft-deletes `Content` and auto-resolves its open reports.

8 Anticipated Q&A

8.1 Tech stack & architecture

Q: What's the tech stack?

A: MySQL 8.x, Node.js + Express, `mysql2/promise` with connection pooling. See Section 2 for the full table and the "why" for each layer.

Q: Why Express and not [some other framework]?

A: Fixed by the course track. Within that constraint, routes stay thin (map HTTP verb+path to a controller) and controllers hold the actual query/procedure-call logic – a standard separation that keeps HTTP concerns out of data-access code.

Q: Why a connection pool instead of one connection?

A: A single shared connection would serialize every request against the database and become a bottleneck under concurrent users; a pool lets multiple requests get their own connection from a

reused set instead of opening/closing a TCP connection per request.

Q: How do you prevent SQL injection?

A: `mysql2` prepared statements / parameterized queries – user input is always passed as a bound parameter (`?`), never string-concatenated into SQL. Same applies to procedure calls (`CALL sp_x(?, ?, ...)`).

8.2 Database design

Q: Why the supertype/subtype (ISA) design instead of one big table or two separate tables?

A: One big table would carry a pile of nullable columns that only apply to one kind of submission. Two fully separate tables would break `Vote`/`Report`, which need a single, honest foreign key target regardless of submission kind. The supertype gives both subtypes a shared identity to be voted on/reported against, while each subtype table only carries the columns relevant to it. See Section 4.2.

Q: What does "disjoint, total" mean?

A: *Disjoint*: a `Content` row is never both an `ExamIntel` and a `Materials` at once. *Total*: every `Content` row *must* be one of the two – there's no "bare" Content row with no subtype. Enforced in practice by always inserting both rows together inside `sp_submit_content`'s transaction.

Q: What's the difference between this and the already-existing SIIT Study Hub product?

A: That's a live Postgres/Supabase app built for a different course (Digital Business Experience, not CSS326). This is a from-scratch MySQL/Node.js redesign for CSS326's required stack – same product idea, reused lessons (e.g. the `CourseOffering` normalization fix), but a deliberately different schema, not a port.

Q: Is the schema normalized? What normal form?

A: Yes – entities are split so that no non-key attribute depends on anything other than the whole key of its own table (3NF in spirit): instructor/semester data lives once in `CourseOffering` rather than repeated per submission; subtype-specific attributes (exam format vs. markdown body) live only on the table they apply to, not as nullable columns on a shared table.

Q: Why do `ExamIntel` and `Materials` share their primary key with `Content` instead of having their own auto-increment ID?

A: This is exactly how supertype/subtype (table-per-subtype / class-table inheritance) is implemented relationally: the subtype's PK *is* the supertype's PK, also declared as a FK back to `Content`. It guarantees a 1:1 relationship between a `Content` row and its subtype row – there's no way to create an `ExamIntel` row that doesn't correspond to exactly one `Content` row.

8.3 Procedures, triggers & transactions

Q: What's the difference between a stored procedure, a function, and a trigger, and where did you use each?

A: A **procedure** is explicitly called by the app (`CALL sp_x(...)`) to perform an action, optionally returning nothing or a result set – used for `sp_register_user`, `sp_submit_content`,

`sp_moderate_content`. A **function** is called inline within a query/expression and returns a single value – used for `fn_course_summary`. A **trigger** fires automatically on an INSERT/UPDATE/DELETE event, with no explicit call from the app – used for the three triggers in Section 5.1.

Q: What happens if `sp_submit_content` fails halfway through (e.g. the subtype insert fails after the Content insert succeeded)?

A: It can't leave a half-written row: both inserts happen inside one `BEGIN/COMMIT/ROLLBACK` block, so if the second insert fails, the whole transaction rolls back and the first insert is undone too. That's the entire reason it's a transaction and not two separate `INSERT` statements from the app.

Q: What happens if two users vote on the same content at nearly the same time – any concurrency concerns?

A: The `Vote(content_id, user_id)` unique constraint stops the same user from voting twice on the same content (a second insert attempt fails/upserts instead of duplicating). `trg_vote_score` recalculates the denormalized score from the actual `Vote` rows on every insert/delete, so even under concurrent votes from *different* users, the trigger fires per-row and the score stays consistent – it's not a read-modify-write race in application code.

Q: Why can't the "Materials needs body text or a file" rule be a table `CHECK`?

A: Because MySQL's `CHECK` constraint can only reference columns within the *same* table, and the two columns involved – `Content.file_path` and `Materials.body_md` – live in different tables after the supertype/subtype split. The rule moved into `sp_submit_content` instead. See Section 4.5.

Q: Why soft-delete instead of an actual `DELETE`?

A: Moderation needs to be reversible (`sp_moderate_content` can restore content) and auditable (who removed it, when, why). A hard `DELETE` would destroy that information permanently.

8.4 Security

Q: How are passwords stored?

A: Never in plaintext. `sp_register_user` generates a random per-user salt (`VARBINARY`), hashes the salted password, and stores only `password_hash` + `salt` – both set exclusively by that procedure.

Q: How do you stop a non-admin from calling the moderation endpoint directly?

A: `sp_moderate_content` checks the caller's `is_admin` flag *inside the procedure itself* before taking any action – so even if a request reached the database layer through a path that skipped an app-level check, the procedure still refuses the action. Same "defense in depth" reasoning as Section 3.1.

8.5 Scope & what's not done yet

Q: Is the JSON/NoSQL (X DevAPI) part finished?

A: No, and that's intentional – it's a Week 12 syllabus milestone. The `JSON` column type for `format/topics` is already decided in the physical design; querying it via the document-store API comes later. Framing: "we know exactly what we don't know yet" is a legitimate proposal-stage answer.

Q: What's actually built right now, at the proposal stage, vs. planned?

A: Built: the full ER design, the 9-table schema, the physical design (data types, keys, constraints), and the reasoning behind every procedure/trigger. Not yet built: the actual Express routes/controllers, the procedures/triggers as executable SQL, and the frontend – those land Weeks 9–13 per the timeline in Section 9.

8.6 Team & process

Q: Who did what?

A: Touch-ing Yinyong – database design lead (ER diagram, schema, normalization decisions, physical design, this proposal). Pinyapach Sinlapakantang – backend, security & triggers (Express routes, `mysql2` integration, the four procedures/functions, the three triggers, auth, transactions). Yayika Tinner – frontend, UI & testing (course pages, submission forms, moderation queue view, transaction/concurrency test cases).

9 Work Distribution & Timeline

Area	Owner	What it covers
Database design & docs	Touch-ing Yinyong	ER diagram, normalization, physical design table, keeping schema in sync, this document and the proposal.
Backend, triggers & security	Pinyapach Sinlapakantang	Express routes, `mysql2` queries, 4 procedures/functions, 3 triggers, password hashing, privilege checks, transaction boundaries.
Frontend, UI & testing	Yayika Tinner	Course catalogue, course page, submission forms, moderation queue view, transaction/concurrency test cases.

Week	Date	Milestone
7	Sept 17	Project proposal presentation (this document supports it)
8	Sept 24	Midterm exam week – no project work scheduled
9	Oct 1	Stand up the full schema (all 9 tables); basic Express CRUD for Course/Content
10	Oct 8	Implement `sp_submit_content` as a transaction; auth via `sp_register_user`
11	Oct 15	Implement the three triggers, `sp_moderate_content`, DB privilege hardening
12	Oct 22	JSON / X DevAPI module – `format/topics` as JSON, document-store queries
13	Oct 29	Revision week – polish UI, write test cases, fix bugs
14	Nov 5	Project demonstration
15	Nov 12	Final exam

10 Glossary (fast lookup if a term catches you off guard)

- **ISA / supertype-subtype**: modeling one entity (`Content`) as a general case with specialized variants (`ExamIntel`, `Materials`) that share the supertype's primary key.
- **Disjoint / total**: disjoint = subtypes don't overlap; total = every supertype row has a subtype.
- **ACID**: Atomicity, Consistency, Isolation, Durability – the guarantees a transaction provides. `sp_submit_content`'s transaction is specifically about *atomicity*.
- **Normalization**: organizing tables so each fact is stored once, avoiding update anomalies (the `CourseOffering` fix in Section 4.4 is a normalization example).
- **Denormalization**: deliberately storing a derived/duplicated value for read performance – the vote score on `Content`, kept in sync by `trg_vote_score`.
- **ENUM**: a MySQL column type restricted to a fixed list of string values.
- **Connection pooling**: reusing a set of open database connections across requests instead of opening a new one per request.
- **Prepared statement / parameterized query**: SQL with placeholders (`?`) bound to values separately from the query text – the standard SQL-injection defense.
- **X DevAPI**: MySQL's NoSQL-style document-store API, letting JSON columns be queried collection-style rather than purely relationally.

Last check before walking in: can each of the three of us explain Section 4 (why the supertype/subtype design) and Section 5 (why each trigger/procedure exists) in our own words, not just read it off this page? That's the part most likely to get a direct follow-up question.